FINAL YEAR PROJECT

Cloud-Based Network Intrusion Detection System (NIDS)

Arrhat Maharjan

Bachelor's in Computing science

Griffith College Cork

8th June 2025

Supervisor: Avinash Nagarajan

1. Abstract

This project aims to design and implement a C**loud-Based Network Intrusion Detection System (NIDS)** using Amazon Web Services (AWS) to address the increasing need for scalable, real-time, and cost-effective cybersecurity solutions. With the exponential growth of cyber threats and cloud adoption, traditional on-premises NIDS solutions often fall short in flexibility, scalability, and manageability. This project leverages cloud-native technologies to build a modular, event-driven NIDS pipeline that provides insights into the network traffic to help determine potential threats in real-time.

The system emulates network traffic using **tcpreplay** on an **AWS EC2 instance**, replaying the **CIC-IDS2017 dataset** to simulate various types of traffic. This traffic is analyzed using **Suricata**, an open-source intrusion detection engine, which generates JSON-based logs. These logs are automatically pushed to **Amazon S3** using a **Python** script, which triggers an **AWS Lambda** function on **Python** that filters, transforms, and forwards the alert logs.

This function forwards the data to **Amazon OpenSearch,** which is used to help us visualize the data, enabling fast and intuitive analysis of security events. Additionally, **Amazon SNS** is integrated to send near real-time notifications to the user.

The project serves as a functional prototype demonstrating the feasibility of building a cost-efficient intrusion detection pipeline using AWS Free Tier services. It emphasizes modularity, scalability, and automation through the integration of multiple cloud tools that AWS provides. Key achievements include developing end-to-end, event-driven architecture, showcasing near real-time alerting and traffic visualization. By utilizing serverless computing (Lambda) and managed services, the prototype minimizes operational overhead and provides a foundation for future scaling and enhancement.

2. Introduction

2.1 Project Background

In today's digital world, network security is a critical concern for organizations of all sizes. As cyber threats continue to evolve in complexity and scale, traditional on-premises based security approaches are no longer sufficient to detect and prevent malicious activities effectively. Intrusion Detection Systems (IDS) play an important role in modern cybersecurity infrastructure by monitoring network traffic and identifying suspicious behaviour.

The motivation behind this project comes from the growing demand for lightweight and scalable security solutions that can operate in real-time across distributed environments. Organizations are increasingly migrating to cloud infrastructures, where traditional on-premises monitoring tools are often inefficient and require an additional personnel, maintenance and iterative enhancements. Small-sized enterprises may lack the resources to deploy and manage robust on-site IDS systems. There is a need for an intrusion detection approach that is both cost-efficient and cloud-native, capable of operating with minimal overhead while also maintaining a high detection accuracy.

This project lays down a prototype that demonstrates what can be accomplished with minimal resources, specifically operating within AWS Free Tier limits, while still delivering real-time detection, automated alerting, and visual analytics. It highlights that, with open-source tools like Suricata and serverless AWS services, it's possible to replicate core IDS capabilities commonly found in commercial offerings, without requiring the substantially more infrastructure and recurring costs those solutions require.

The methodology adopted for this project includes requirements analysis, system design, prototyping, iterative development, and testing. The project emphasizes serverless computing, automation, and real-time processing to ensure scalability, cost efficiency, and low operational complexity. It serves as a functional prototype and proof-of-concept for organizations looking to adopt cloud-native IDS solutions without significant infrastructure investment.

2.2 Technical Background

2.2.1 Introduction to Intrusion Detection

Intrusion Detection Systems (IDS) are vital components of a modern cybersecurity infrastructure. Their primary role is to monitor, log, and analyze network or system activities to detect compromises, anomalies, unauthorized access, and deviations from everyday normal behavior. IDS solutions provide critical visibility into potential threats and are a key element in layered defense strategies.

IDS tools operate in two fundamental ways: **passively**, by generating alerts and logs for human or automated review; and **actively**, by integrating with Intrusion Prevention Systems (IPS) that can actively block malicious activity. As cyberattack methods evolve to include advanced threats, high volume and zero-day vulnerabilities, IDSs have also evolved with improved detection algorithms, threat intelligence integration, and machine learning enhancements.

2.2.2 Cloud Computing and Security

Cloud computing has changed the IT infrastructure industry by offering on-demand computing resources, scalability, and reduced operational overhead. However, this change introduces new challenges in visibility, control, and new attack vectors. Traditional IDS tools were designed for static, centralized networks and are not well-suited for the dynamic nature of cloud environments.

Serverless architecture, such as **AWS Lambda**, allows functions to run in response to events (e.g., a new log file being written). Coupled with **Simple Notification Service (SNS)** for alerting and **Amazon S3** for storage, cloud-native security systems can automate threat detection workflows and reduce time and expenses to users.

2.2.3 Categories of Intrusion Detection Systems

Intrusion Detection Systems can be categorized based on the location of deployment and the method of detection:

Network-based IDS (NIDS)

NIDS analyze packets as they travel across the network, offering broad visibility across multiple hosts and services. They are well-suited for detecting distributed attacks such as

DDoS, reconnaissance, and man-in-the-middle (MITM) attempts. Suricata, as used in this project, is a popular NIDS engine.

Host-based IDS (HIDS)

HIDS are installed on individual machines and monitor internal logs, system calls, and file changes. They provide deep insights into host-specific behaviors but are harder to manage at scale, especially in dynamic cloud environments.

Hybrid IDS

Combining both HIDS and NIDS offers the advantages of broad network visibility and detailed host-level monitoring. While not the main objective of this project, hybrid systems are often implemented in enterprise security designs.

2.2.4 Detection Methodologies

IDS engines can employ different detection models, each with its strengths and trade-offs:

Signature-based Detection

This technique relies on known attack patterns, such as specific byte sequences, protocol violations, or command patterns. Signatures are updated frequently via threat intelligence feeds (e.g., Emerging Threats). While effective against known threats, signature-based systems are blind to novel or obfuscated attacks.

Anomaly-based Detection

Anomaly detection builds a statistical or machine learning model of normal behavior, flagging deviations as potential threats. It excels at detecting unknown or evolving attacks, such as zero-days. However, it can suffer from higher false positives and requires robust training data and tuning.

Specification-based Detection

In this method, analysts manually define acceptable behavior for applications or protocols. Any deviation from the expected flow is flagged as anomalous. It blends accuracy with flexibility but is labor-intensive and requires domain expertise.

2.2.5 Suricata Intrusion Detection Engine

Suricata is an open-source intrusion detection and prevention system (IDS/IPS) developed by the Open Information Security Foundation (OISF). It is a versatile and powerful tool that supports various operating systems including Windows, macOS, Unix, and Linux. Suricata can analyze live network traffic or replaying pre-recorded traffic from **Packet Capture (.pcap)** files, which is how it's utilized in this project using **tcpreplay**.

Suricata is lightweight, cost-effective, and capable of providing detailed insights into network activity, making it an excellent choice for both small and large-scale deployments. Its multi-thread nature offers an advantage over alternatives like Snort. Its supports Snort rules and specialized Suricata rulesets, allowing reuse of community-curated signatures. The eve.json format is structured and machine-readable, ideal for integration with tools like AWS Lambda and OpenSearch.

2.2.6 Cloud-native Intrusion Detection Architecture

Cloud-native IDS is designed to operate natively in cloud environments without reliance on traditional infrastructure. It emphasizes **automation**, **event-driven computing**, and **pay-per-use pricing**.

This project demonstrates a simplified version of such architecture using:

- **EC2 instance**: Hosts Suricata and replays traffic using tcpreplay.
- **S3 bucket**: Stores log files (eve.json) for further processing.
- **AWS Lambda**: Reads new logs, filters for alerts, and performs lightweight analysis.
- **SNS**: Sends real-time alerts to subscribers (e.g., emails or SMS).
- **OpenSearch**: Provides dashboards for visualizing logs, trends, and alerts.

This pipeline eliminates the need for manual monitoring and scales horizontally as needed. Moreover, staying within the AWS Free Tier ensures affordability and replicability for SMEs and researchers.

2.2.7 CIC-IDS2017 Dataset

The CIC-IDS2017 dataset, put together by researchers Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani of the Canadian Institute for Cybersecurity (CIC), is one of the most comprehensive and realistic publicly available datasets for evaluating intrusion detection systems. It was designed to address the limitations of older datasets and offers

up-to-date attacks, diverse traffic patterns, accurate labels, and full network captures, making it highly suitable for both academic research and production IDS evaluation.

The dataset features realistic user traffic with activities like browsing, email, file transfers, and streaming, using various protocols. It includes diverse attack types such as brute force, DoS, infiltration, web attacks, and botnets. It provides over 80 features per flow (e.g., duration, packet size, protocol, flow statistics), and is available in both .pcap and .csv formats for different analysis methods.

In this project, .pcap files from CIC-IDS2017 are replayed through a virtual network interface using tcpreplay which, Suricata, configured on an EC2 instance, captures and analyzes the traffic.

2.2.8 Amazon Web Services

Amazon EC2
EC2 is used to host Suricata and replay traffic using tcpreplay. It provides a virtual compute environment with configurable resources and network settings, allowing flexible deployment of IDS components.

Amazon S3
S3 serves as the centralized storage for Suricata logs in JSON format. It enables scalable, durable, and secure object storage, and supports event notifications to trigger downstream processing.

AWS Lambda
Lambda functions run serverless Python code automatically when new logs arrive in S3. This enables real-time, event-driven processing of logs without the need for always-on infrastructure.

Amazon SNS
SNS is used to send real-time email notifications when high-severity alerts are detected. It integrates seamlessly with Lambda to ensure timely alerting.

Amazon OpenSearch Service
OpenSearch stores and indexes the processed logs, offering a dashboard interface for querying, filtering, and visualizing network activity and alerts.

IAM (Identity and Access Management)
IAM is used to securely manage access between services. Each component is assigned specific roles and policies to follow the principle of least privilege.

Security Groups
Security groups are configured to control inbound and outbound traffic to the EC2 instance, ensuring only trusted sources can access necessary ports (e.g., SSH or OpenSearch).

2.3 Scope and Objectives

Primary Objectives

The primary goal of this project is to design and deploy a cloud-based Network Intrusion Detection System (NIDS) on AWS, capable of detecting malicious traffic through automated log analysis. Key objectives include:

- Deploying Suricata for deep packet inspection and rule-based intrusion detection.
- Emulating realistic network traffic using tcpreplay to test detection capabilities.
- Automating log ingestion, processing, and alerting using Python scripts, AWS Lambda, and SNS.
- Visualizing security data using **OpenSearch dashboards**.
- Ensuring modularity, scalability, and cost-effectiveness within the AWS Free Tier.

Success Criteria

The project's success will be measured against the following KPIs:

- **Detection Accuracy**: Suricata should detect predefined threats from the CIC-IDS2017 dataset.
- **Alert Responsiveness**: Notifications via SNS should be triggered within 5 seconds of log detection.
- **System Stability**: EC2 instance and Lambda functions should demonstrate consistent uptime during tests.
- **Resource Efficiency**: Total AWS usage should remain within the Free Tier limits.
- **Dashboard Usability**: OpenSearch dashboards must provide real-time log visibility and custom filters.

Limitations

The scope is limited to prototyping and demonstration. As such:

- **Live traffic monitoring** is not included; traffic is replayed from existing PCAP files.
- **Free Tier constraints** limit compute power, storage, and service usage.
- **Threat Prevention** (e.g., firewall response) is not part of this implementation.
- **OpenSearch Stability** may be affected due to the AWS free tier constraints, as only 1 node is being used.

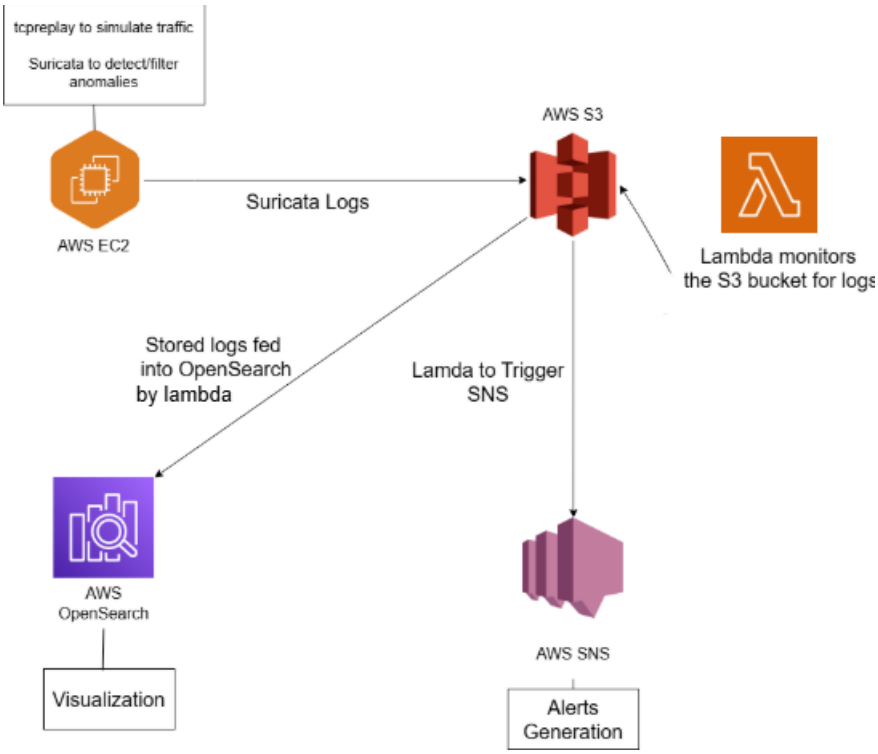
This scope ensures the project remains manageable while delivering a working prototype that demonstrates key NIDS features on the cloud.

3. Architecture and System Design

3.1 Architecture Overview

The cloud-based NIDS system follows a modular, event-driven architecture that integrates multiple AWS services to enable near real-time intrusion detection, alerting, and traffic analysis. The core architecture consists of five primary components: the traffic simulation component (tcpreplay and PCAP files), the detection layer (Suricata IDS), the processing layer (Python scripts and Lambda functions), the storage layer (S3 and OpenSearch), and the presentation layer (SNS notifications and OpenSearch Dashboards).

IAM Roles and Policies that the AWS provides are used to ensure a secure and limited access between services.



3.2 Technical Architecture

Infrastructure Components

The technical architecture leverages AWS services to provide enterprise-grade reliability, security, and performance. The primary compute resource is an EC2 instance configured with appropriate size to handle the expected traffic replay and detection workloads. The instance is deployed within a Virtual Private Cloud (VPC) with carefully configured security groups and network access control lists (NACLs) to ensure proper network isolation and security.

Storage architecture utilizes Amazon S3 for durable, scalable log storage with configurable lifecycle policies to optimize costs. The S3 bucket is configured with versioning, encryption, and cross-region replication to ensure data durability and compliance with security requirements. Event notifications are configured to trigger downstream processing automatically when new log files are uploaded.

Data Flow Architecture

Data flows through the system in a unidirectional pattern from traffic generation through detection, processing, storage, and presentation. Each stage in the pipeline operates independently with clear input and output specifications, enabling independent scaling and maintenance of individual components.

- **Traffic Replay:** The EC2 instance replays pcap files using tcpreplay to simulate real network activity.
- **Detection:** Suricata, running on EC2, monitors the traffic and writes alert logs into an eve.json file.
- **Log Upload:** A python script monitors the file constantly and uploads these logs to an Amazon S3 bucket in batch using the **boto3** AWS SDK. The script can be configured to get the desired the upload interval and number for logs per batch to upload the logs to the S3 bucket, which are stored as jsonl files.
- **Trigger & Processing:** S3 triggers an AWS Lambda function, a python script, upon log arrival. The function parses the log entries, identifies high severity alerts, invokes notification and forwards the logs to OpenSearch, also using the **boto3** SDK.
- **Notification:** If critical threats are identified, Lambda invokes SNS to send email alerts to subscribed users.
- **Visualization:** All logs are indexed into OpenSearch, where OpenSearch Dashboards provides filtering, querying, and visual analytics.

Security Architecture

- **IAM Roles:** Fine-grained roles prevent unnecessary privileges; each AWS service has the access needed only.
- **VPC Isolation:** EC2 is deployed in a private subnet to limit exposure.
- **Encrypted S3 Buckets:** Ensures log data is encrypted.
- **Environment Variables in Lambda:** Secrets and keys are not hardcoded.
- **Access Control:** The only public accessible component is OpenSearch, and access to it is user authenticated.

3.3 Design Principles

Cost Optimization

The system prioritizes cost optimization through multiple approaches to maximize value while minimizing expenses. These strategies have been validated to keep the entire system within AWS Free Tier limits for small to medium deployments while providing clear scaling paths for larger implementations.

- **EC2 Instances**: Primary deployment uses a t2.micro instance (750 hours/month free tier) with a 1 GB of Memory and 1 CPU core. The instance has been configured to use an additional 2 GB swap memory.
- **S3 Storage**: Lifecycle policies keep storage costs within the 5GB free tier limit by automated deletion policy for logs exceeding retention requirements.
- **AWS Lambda**: Provides as serverless execution with zero cost during idle, only being executed on the trigger event.
- **Amazon SNS**: Topic-based notification system ensures use is limited to the invoke from the Lambda.
- **Event-Driven Processing**: Eliminates the need for continuously running services.

Monitoring and Observability

The monitoring strategy uses a of AWS-native tools for system information and OpenSearch service to get operational insight and traffic data insight.

- **CloudWatch Logs**: All Lambda function executions are logged to AWS CloudWatch, capturing output, errors, and performance metrics such as execution

time, memory usage, and invocation counts. These logs help identify bottlenecks, exceptions, and anomalies.

- **CloudWatch Metrics**: Metrics provides insight into the resources being used, from network information in the EC2 instance to the count notifications delivered by SNS.
- **Billing Alerts**: To remain within AWS Free Tier limits and avoid unexpected charges, CloudWatch billing alarms are configured. These alarms notify the user when estimated charges approach predefined thresholds.
- **OpenSearch Dashboards and Analytics**: Network traffic logs and security alerts ingested into OpenSearch are visualized using OpenSearch Dashboards. This provides insights through filters, queries, and time-series charts to identify trends, anomalies, or suspicious behaviours.

Scalability Considerations

Scalability is a key design principle in the system to ensure that it can adapt to higher workloads and future enhancements without major architectural changes.

- **Serverless Lambda**: AWS Lambda automatically scales with the volume of incoming log files. Each trigger invokes a separate instance of the function, allowing parallel processing.
- **S3 Storage**: Amazon S3 offers virtually unlimited storage and automatically scales with the amount of data uploaded. This makes it ideal for storing large volumes of network logs over time, with no infrastructure management.
- **OpenSearch Clustering**: OpenSearch can be scaled horizontally by adding more data nodes or coordinating nodes. This enhances performance for large-scale indexing and supports high-throughput analytical queries.

4. Component Deep Dive

4.1 EC2 Instance Configuration

This section details the configuration of the Amazon EC2 instance used in the cloud-based Network Intrusion Detection System (NIDS). The EC2 instance hosts Suricata for network traffic analysis and uses tcpreplay to simulate a real-world network scenario using packet capture (.pcap) files. A Python script monitors logs and uploads them to Amazon S3 for further processing.

4.1.1 Infrastructure Specifications

4.1.2 tcpreplay Implementation

Configuration Parameters

- **-i [string], i--intf1=eth0:** Outbound interface for replay
- **-p [string], --pps=1000:** Replay speed in packets per second
- **-l [number], --loop=10:** Loop count
- **-t**, **--topspeed:** Replay packets as fast as possible
-

PCAP File Management

- PCAP files from the CIC-IDS2017 dataset are stored under /home/ec2-user/pcap_files/
- Files are separated into two halves for each day.

Command-Line Usage Examples

```
#normal speed
tcpreplay –i tap0  /home/ec2-user/pcap_files/thurs-part1.pcap
# top speed for the interface
tcpreplay –i tap0 -t /home/ec2-user/pcap_files/thurs-part1.pcap
```

4.1.3 Suricata IDS Setup

Installation and Initial Configuration

The main file for configuration is the suricata.yaml file it generates on installation. All changes and optimization are configured using the file.

Performance Optimization

- Supressed checksum validation for specific IPs (e.g., Windows Update server 13.107.4.50) to avoid false positives.
- Increased Suricata's packet buffer size for af-packet to handle higher traffic volumes efficiently.
- Enabled memory-mapped file I/O (mmap) for Suricata to improve performance by reducing disk I/O overhead.
- Enabled tpacket_v3 kernel feature to use an efficient ring buffer for packet capture, enhancing throughput.

Detection Engine Tuning

- Increased max-pending-packets to 1024 to handle bursts of network traffic.
- Configured HOME_NET to include internal subnets and known CDN servers to reduce noise.
- Suppressed IPv4 checksum validation on specific hosts to reduce false positives.

Log Format Configuration

EVE.JSON output format is selected for its compatibility with AWS Lambda and OpenSearch. As the goal to detect anomalies, the logging for TLS, SMTP, HTTP and others are disabled. The logging like solely based on the rules built.

Rule Set

A custom.rules file is used as the rules for the engine. The default rules are disable.

The rules are made with the target to detect the various threats the data set comprises.

Example rule:

```
# format:
# action protocol src_ip src_port -> dst_ip dst_port (options)

alert tcp any any -> 192.168.1.10 80 (msg:"Possible HTTP Attack";
flow:to_server,established; content:"/admin"; http_uri; sid:1000001; rev:1
```

This rule triggers an alert when **any TCP packet going to 192.168.1.10 on port 80** contains the string /admin in the HTTP URI, which might indicate someone trying to access the admin page, possibly suspicious or unauthorized activity.

For more on rules: https://docs.suricata.io/en/latest/rules/index.html

4.1.4 Python Log Processing Script

Script Architecture and Design

A modular Python script continuously monitors Suricata's eve.json log file. It batches new log entries, adds a processing timestamp, and uploads them to an AWS S3 bucket in JSON Lines (.jsonl) format using the **boto3** AWS SDK. Each upload is metadata-tagged and timestamped for traceability.

File Monitoring Implementation

The script uses position-based tracking to detect new log entries. It reads and processes only the appended lines and supports log rotation by resetting the position if the file size shrinks.

Batching and Upload Logic

Logs are grouped into batches of 200 entries or uploaded every five minutes, whichever occurs first. These can be configured to address the type of logging the user desires. Batches are named using timestamps and include metadata like batch size and upload time. Batches are uploaded to the suricata-logs/ prefix in S3. Between each upload the code is put on hold for 15 seconds.

S3 Upload and Metadata

Each batch is uploaded via put_object() with relevant metadata, such as batch id, metadata and content type (application/json). Metadata also includes a timestamp and format specifier (jsonl).

Error Handling and Retry Logic

Robust error handling is built in using try-except blocks. Any JSON decoding or upload failures are logged locally using Python's logging module. Upload errors include messages in both the console and s3_upload.log.

Logging and Monitoring

All significant actions, such as batch uploads, line processing counts, and errors, are logged to both stdout and s3_upload.log. The system reports the total number of entries uploaded upon shutdown.

4.2 AWS S3 Storage Layer

Amazon S3 acts as the centralized and persistent storage layer for logs generated by the . The system is designed to automatically ingest, store, and trigger processing events based on file uploads. This section outlines the configuration of the S3 bucket and its integration with event-driven processing mechanisms.

4.2.1 Bucket Configuration

Log files are stored using a structured path 'suricate-logs/{batch_id}.jsonl'. **Lifecycle policy** is configured to delete logs after 14 days (for cost control). IAM policy grants upload access only to EC2 instance and Lambda function for processing. **Server-side encryption (SSE-S3)** is enabled by default, that is provided by AWS. All objects are encrypted automatically with AWS-managed keys, this ensures data confidentiality without needing additional setup.

4.2.2 Event Notifications

The S3 bucket is configured to trigger event notifications to AWS Lambda when new logs are uploaded. This triggers the next stage in the pipeline for real-time alerting and visualization. As only the Suricata logs are being uploaded, this triggers lambda-sns function on any object creation event. Lambda processes the JSON log and extracts relevant alert metadata. Events are filtered to only trigger on .jsonl files under suricata-logs/ prefix preventing unnecessary executions on non-log file.

4.3 Lambda Function Processing

4.3.1 Function Architecture

The AWS Lambda function serves as the core processing engine for security log analysis, alert generation, and data ingestion. The function runs on Python 3.14 with a memory allocation of 124 MB and a timeout of 3 seconds to balance cost and performance. Environment variables are used to configure the S3 bucket, OpenSearch endpoint, OpenSearch authentication and SNS topic ARN. Third party **opensearch-py,** a wrapper for OpenSearch REST API**,** is packaged into a Lambda layer to reduce deployment size and improve cold start times.

4.3.2 Log Parsing Logic

The function parses each .jsonl object from S3 using line-by-line JSON decoding. Relevant fields such as timestamp, alert message, source/destination IPs, and protocols are extracted and reformatted. Invalid or corrupt entries are skipped with detailed error

logging. To optimize performance, logs are processed in-memory, and only essential fields are passed forward to minimize payload size.

4.3.3 Integration Points

After parsing, critical alerts are published to an SNS topic in a human-readable format to notify subscribers in near real-time. Parsed data is also indexed into OpenSearch using bulk ingestion. Any processing or integration errors are logged using CloudWatch Logs for monitoring and debugging.

4.4 SNS Notification System

4.4.1 Topic Configuration

An SNS topic (s3-event) was created to publish intrusion alerts. Message filtering is applied by Lambda using a predefined attribute 'metadata.severity.keyword', ensuring only relevant alerts reach subscribers. The message invoked using the **boto3** AWS SDK.

4.4.2 Alert Formatting

Alerts follow a structured template including time, alert type, IPs, and protocol. Severity levels (low to critical) guide routing: critical-severity alerts trigger SNS notifications, while low/medium alerts are logged only. This formatting ensures clear, relevant, and prioritized notifications.

Example for high alert:

```
        message = f"""
{severity_emoji} SECURITY ALERT - Suricata Detection

🔍 SEVERITY: {severity.upper()}
🚨 Signature: {alert_info.get('signature', 'Unknown')}
📋 Category: {alert_info.get('category', 'Unknown')}
🆔 Rule ID: {alert_info.get('signature_id', 'Unknown')}
⚡ Action: {alert_info.get('action', 'Unknown')}

Network Details:
• Source: {log_data.get('src_ip', 'Unknown')}:{log_data.get('src_port', 'Unknown')}
• Destination: {log_data.get('dest_ip', 'Unknown')}:{log_data.get('dest_port', 'Unknown')}
• Protocol: {log_data.get('proto', 'Unknown')}
• Interface: {log_data.get('in_iface', 'Unknown')}
• Flow ID: {log_data.get('flow_id','Unknown')}

File Details:
• Bucket: {bucket_name}
• File: {object_key.split('/')[-1]}
• Time: {formatted_time}
• Size: {object_size} bytes

Full Path: s3://{bucket_name}/{object_key}
```

4.5 OpenSearch Analytics Platform

OpenSearch Dashboards provide analytics and visualization of parsed intrusion logs. It connects directly to the OpenSearch domain by indexing Suricata alerts.

4.5.1 Cluster Configuration

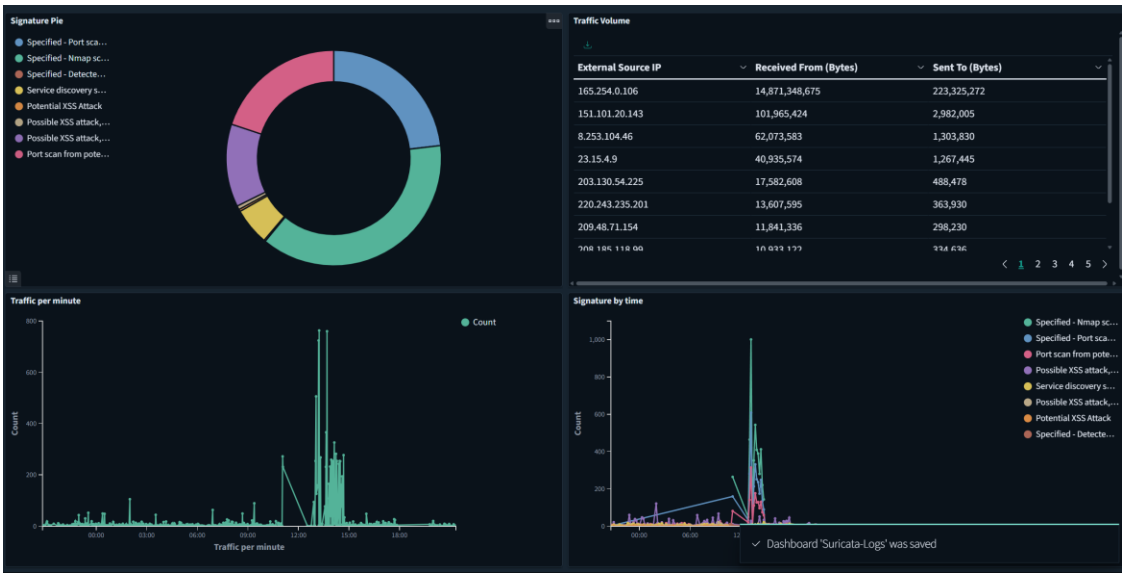
The prototype uses a **single-node development cluster and t3.small.search instance** under the AWS Free Tier, which is suitable for low-ingest volumes. Suricata logs are stored in a dedicated index: suricata-logs-*. OpenSearch automatically maps the field of the JSON data that's ingested. Basic authentication enabled for OpenSearch Dashboards, with username and password. IAM-based access control implemented for indexing rights from the Lambda ingestion layer.

4.5.2 Data Ingestion

Ingested via Lambda, which reads Suricata logs from S3, parses JSONL, and writes to OpenSearch using HTTPS endpoint, as an individual JSON object. Depending on the log ingestion script at the EC2, the ingestions can be instant, but to remain under the free tier limits that is not possible.

4.5.3 Visualization and Dashboards

OpenSearch Dashboards is used to visualize and monitor intrusion detection data collected by Suricata. It provides interactive dashboards that display key metrics such as top alerts, source and destination IPs, protocol distribution, and alert severity over time. Using index patterns created from the index, the system enables filtering, time-based analysis, and querying. These visualizations help users quickly identify trends, investigate anomalies, and respond to potential threats more effectively.



5. Implementation Guide

This section provides a walkthrough of the implementation process for the project. It covers setup and configuration.

5.1 EC2 Instance:

Launch an EC2 instance (Amazon Linux), connect to it with the CloudShell on the AWS console or connect via SSH. SSH access needs an inbound rule on port 22.

Since we are using a virtual interface to replay the traffic, to setup the tap0 interface, use the following command:

```
sudo ip tuntap add dev tap0 mode tap

sudo ip link set dev tap0 up

sudo ip addr add 192.168.200.1/24 dev tap0
```

If using free tier, it is recommended to configure a swap space for the machine to be able to gain access to a virtual memory.

```
sudo dd if=/dev/zero of=/swapfile bs=1M count=2048
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
# verify
free -h

# Make it permanent
sudo nano /etc/fstab
/swapfile none swap sw 0 0
```

Required Software

After setting up the EC2 instance and swap, the next step is to install the necessary software components: tcpreplay for replaying network traffic and Suricata as the Network Intrusion Detection System (NIDS).

To install tcpreplay:

```
# prerequisites for installation
sudo yum groupinstall "Development Tools"
sudo yum install libpcap-devel

wget https://github.com/appneta/tcpreplay/releases/download/v4.5.1/tcpreplay-4.5.1.tar.xz

tar –xf tcpreplay-4.5.1.tar.xz

cd tcpreplay-4.5.1.tar.xz

./configure

make

sudo make install

# to test run
sudo make test
```

Official installation guide: **https://tcpreplay.appneta.com/wiki/installation.html**

To install Suricata:

```
#minimul dependencies/prerequisites
sudo dnf install -y gcc libpcap-devel libnet-devel pcre-devel libyaml-devel file-devel zlib-devel jansson-devel nss-devel libcap-ng-devel libmaxminddb-devel libnetfilter_queue-devel lua-devel lz4-devel rustc cargo

wget https://www.openinfosecfoundation.org/download/suricata-7.0.10.tar.gz
tar xzvf suricata-7.0.4.tar.gz
cd suricata-7.0.4
./configure
make
make install
```

Official install guide: https://docs.suricata.io/en/latest/install.html

To add .pcap file to the instance use the **wget** tool to download the file from the internet.

```
wget {Download link}
```

File transfer from local machine to the instance and vice versa can be done with the **scp(Secure Copy Protocol)** tool available in Linux, which uses SSH protocol, but can be dependent on the upload speed.

If using custom rules add the path to the ‘rules-file’ tag in the suricata.yaml file.

```
rule-files:
#  - suricata.rules
  - /home/ec2-user/custom.rules
```

Add the interface to ‘af-packet’ tag:

```
af-packet:
  - interface: lo
```

Add the IP for the internal machines to ‘address-groups’ :

```
vars:
  # more specific is better for alert accuracy and performance
  address-groups:
    HOME_NET: "[192.168.10.0/24, 205.174.165.66, 205.174.165.68, !$KNOWN_GOOD_IPS]"
```

For more configuration option for the 7.0.4 version (current stable):

https://docs.suricata.io/en/suricata-7.0.4/

Now we can replay the traffic and detect with Suricata.

```
# replay a pcap file
sudo tcpreplay -i tap0 {name}.pcap
# to start detection
sudo suricata -i tap 0 -c /usr/local/etc/suricata/suricata.yaml
# use tcpdump to verify traffic replay
sudo tcpdump -i tap0
```

You will need multiple terminals connected to the instance.

Develop a script that enables uploads to S3 bucket. If you use python, using boto3:

```
s3_client = boto3.client('s3')

 # upload to S3
s3_client.put_object(
    Bucket=BUCKET_NAME,
    Key=object_key,
    Body=body,
    ContentType=content_type,
    Metadata=metadata
)
```

This shows the initialization of the object reference for a S3 service and the function to PUT and object.

5.2 S3 setup

Create an S3 bucket to store Suricata logs and alerts. For the AWS Free Tier, the default 5 GB storage is sufficient for low-volume traffic. Enable **versioning,** if available, and **server-side encryption (SSE-S3)** for data durability and security. Set a lifecycle policy to automatically delete logs older than 14 days to save storage costs. Set the prefix and suffix to remove unnecessary processing for the Lambda.

Add an Event Notification for the S3, on the desired condition, we will be using the All object create events (s3:ObjectCreated:*), so that the function is triggered when an object in inserted into the S3 bucket. Add the destination to the Lambda on the next step.

5.3 Lambda

Create a Lambda function, in this case in a Python Runtime. Configure the trigger to the S3 Event Notification. Develop a script to that is able to access the S3 bucket, parse the data and push to the OpenSearch Domain. The library **opensearch-py** can be used to make requests on the REST API that OpenSearch provides.

Add the important variable into the Environment Variables of the function to secure it from any exploit.

Permission for the S3 and the SNS should be added to the function so that it can interact with the services.

In this code using boto3, the object reference for S3 and Environment Variables are initilaized.

```
# initialize S3 client
s3_client = boto3.client('s3')

# environment variable for SNS
SNS_TOPIC_ARN = os.environ['SNS_TOPIC_ARN']

# environment variables for OpenSearch
OPENSEARCH_ENDPOINT = os.environ['OPENSEARCH_ENDPOINT']
OPENSEARCH_INDEX = os.environ['OPENSEARCH_INDEX'] if 'OPENSEARCH_INDEX' in os.environ else 'suricata-logs

#authentication for OpenSearch
OPENSEARCH_USERNAME = os.environ['OPENSEARCH_USERNAME']
OPENSEARCH_PASSWORD = os.environ['OPENSEARCH_PASSWORD']
```

Here the object information is gathered.

```
for record in event['Records']:
    # extract S3 event details
    bucket_name = record['s3']['bucket']['name']
    object_key = unquote_plus(record['s3']['object']['key'])
    object_size = record['s3']['object']['size']
    event_name = record['eventName']
    event_time = record['eventTime']
    region = record['awsRegion']
```

Here the object reference for SNS and OpenSearch is initialized.

```
sns = boto3.client('sns')
```

```
# create OpenSearch client
client = OpenSearch(
    hosts=[{'host': OPENSEARCH_ENDPOINT.replace('https://', ''), 'port': 443}],
    http_auth=(OPENSEARCH_USERNAME, OPENSEARCH_PASSWORD),
    use_ssl=True,
    verify_certs=True,
    connection_class=RequestsHttpConnection,
    timeout=60
)
```

Here the function to publish the SNS and bulk ingest data into the OpenSearch

```
response = sns.publish(
    TopicArn=topic_arn,
    Message=message,
    Subject=subject
)
```

```
# prepare bulk request body
bulk_body = '\n'.join(events) + '\n'

try:
    # execute bulk request
    response = client.bulk(body=bulk_body)
```

5.4 SNS

Create an SNS topic for alert notifications. Use the Standard Type for Email messages. Subscribe endpoints to receive critical alert messages, this can be done on the AWS console. The Lambda function publishes notifications only for alerts meeting a predefined severity threshold to avoid too many alerts. Notifications include structured details: alert timestamp, source and destination IPs, protocol, and alert description, that the lambda passes to SNS.

5.5 OpenSearch

Deploy a single-node OpenSearch cluster (e.g., t3.small.search instance) within AWS Free Tier limits, in a public domain. Secure OpenSearch with IAM-based access policies or basic authentication, basic in this case. Create indices with a naming pattern (e.g., suricata-logs-*) for storing Suricata alert data.The Lambda function ingests logs into OpenSearch using the bulk API for performance. Configure OpenSearch Dashboards to visualize alerts and generate reports on traffic anomalies, frequent alert types, top IP addresses, and severity distribution.

5.6 IAM and Policies

A role with the policy to access the S3 service and the lambda attached to it is attached to the instance, letting it freely interact with the services. Other than for testing, access to Lambda is unnecessary.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:*",
        "s3-object-lambda:*"
      ],
      "Resource": "*"
    }
  ]
}
```

Add a role to Lambda with policies with access to the S3 bucket and the SNS topic.

This can be done in the AWS console or manually.

For SNS:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "SNSFullAccess",
      "Effect": "Allow",
      "Action": "sns:*",
      "Resource": "*"
    },
    {
      "Sid": "SMSAccessViaSNS",
      "Effect": "Allow",
      "Action": [
        "sms-voice:DescribeVerifiedDestinationNumbers",
        "sms-voice:CreateVerifiedDestinationNumber",
        "sms-voice:SendDestinationNumberVerificationCode",
        "sms-voice:SendTextMessage",
        "sms-voice:DeleteVerifiedDestinationNumber",
        "sms-voice:VerifyDestinationNumber",
        "sms-voice:DescribeAccountAttributes",
        "sms-voice:DescribeSpendLimits",
        "sms-voice:DescribePhoneNumbers",
        "sms-voice:SetTextMessageSpendLimitOverride",
        "sms-voice:DescribeOptedOutNumbers",
        "sms-voice:DeleteOptedOutNumber"
      ],
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "aws:CalledViaLast": "sns.amazonaws.com"
        }
      }
    }
  ]
}}
```

For S3:

```
{
   "Version": "2012-10-17",
   "Statement": [
      {
         "Effect": "Allow",
         "Action": [
            "s3:*",
            "s3-object-lambda:*"
         ],
         "Resource": "*"
      }
   ]
}
```

For OpenSearch, in order to be able to use the REST API the domain should have permission to take http request. Add this to access policy for the domain.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": [
          "*"
        ]
      },
      "Action": [
        "es:*",
        "es:DescribeDomain",
        "es:ESHttp*"
      ],
      "Resource": "arn:aws:es:{region}:{user-id}:domain/{domain-name}/*"
    }
  ]
}
```

9. Testing and Validation

9.1 Testing Strategy

Component-wise Testing:
Testing was initially conducted on individual components independently to verify their proper functioning before integration. For example, Suricata's detection capabilities were validated using sample .pcap files on a standalone EC2 instance, tcpreplay was tested to ensure accurate traffic replay, and AWS Lambda functions were verified to execute correctly with test events. This approach helped isolate issues early and ensured that each component was reliable.

Integration Testing:
After individual components passed their tests, integration testing was performed as new components were added and interconnected. This involved validating the complete data flow, from traffic replay on EC2, detection and log generation by Suricata, log upload to S3, triggering Lambda for alert processing, notifications via SNS, and indexing in OpenSearch. Integration tests ensured that components worked together seamlessly, and data integrity was maintained throughout the system.

Automation Framework

Basic automation scripts were developed using Bash and Python to streamline traffic replay, log collection, and result comparison. This reduced error and ensured consistency across multiple test cycles.

9.2 Test Cases

Functional Testing Scenarios
Functional tests were performed first on individual components and later in integrated scenarios. Examples include verifying Suricata's ability to detect known attack signatures, validating the proper replay of network traffic with tcpreplay, ensuring logs were correctly uploaded to S3, Lambda functions executed upon log uploads, SNS notifications were sent, and OpenSearch indexed the logs accurately.

Performance Testing Procedures
Performance testing measured throughput and latency at both component and system levels. For instance, the response times and load capacity of Lambda and OpenSearch services were monitored during integrated operation.

9.3 Validation Procedures

Acceptance Criteria Definition:
Acceptance criteria were defined to cover both component-level and integrated system requirements, including successful detection of known attack patterns, zero packets drop from Suricata, zero loss of log data, timely alert notifications, and stable system performance under expected traffic loads.

Validation Methodology:
Validation included component unit tests, integration testing, automated test execution, and manual review. Results were compared against acceptance criteria, with any failures prompting immediate resolution. Regression testing ensured continued fault fixes.

Quality Assurance Processes:
Quality assurance involved continuous monitoring, error logging, automated alerts, and routine code and documentation reviews to maintain system reliability and robustness throughout development and deployment.

10. Future Enhancements

Short-term Improvements

In the immediate future, enhancements will focus on refining the system's stability and accuracy. Improvements include fine-tuning Suricata rule sets for better threat detection accuracy and configuring or scaling the OpenSearch domain for better results and higher loads.

Long-term Strategic Goals

For the longer term, the project needs to evolve into a more robust and production-ready NIDS platform. Long-term goals include incorporating automated fallback mechanisms in components, response to detected threats, expanding the system's footprint to monitor multiple networks, and enabling redundancy. If possible, be able to go past the free tier limits that AWS imposes.

Technology Evolution Considerations

With the rapid evolution of cloud-native technologies, system architecture must remain adaptable. The roadmap includes evaluating and potentially integrating newer AWS services such as AWS GuardDuty for threat investigation and Amazon EventBridge for improved orchestration. Additionally, integrating Suricata with a Machine learning based thread detection.

11. References

Suricata

https://github.com/daffainfo/suricata-rules/tree/main

https://docs.suricata.io/en/latest/rules/index.html

https://docs.suricata.io/en/latest/output/eve/eve-json-output.html#eve-json-output

https://docs.suricata.io/en/latest/configuration/suricata-yaml.html

AWS

https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-configuration-examples.html

https://docs.aws.amazon.com/lambda/latest/dg/configuration-function-zip.html

https://docs.aws.amazon.com/

https://boto3.amazonaws.com/v1/documentation/api/latest/index.html

https://repost.aws/knowledge-center/ec2-instance-access-s3-bucket

tcpreplay

https://linux.die.net/man/1/tcpreplay

https://www.youtube.com/watch?v=5iK-0MiHZmI&t=141s

https://tcpreplay.appneta.com/wiki/tcpreplay

CIC-IDS2017 Dataset

https://www.unb.ca/cic/datasets/ids-2017.html

OpenSearch

https://www.youtube.com/watch?v=1xaJODdnnOk

https://docs.opensearch.org/docs/latest/

https://docs.opensearch.org/docs/latest/

12. Appendices

Python Script Source Code

```
import boto3
import json
import os
import logging
from botocore.exceptions import ClientError
from datetime import datetime
import time

BUCKET_NAME = 'ids-suricata-logs'
BATCH_SIZE = 200  # number of log entries per batch
BATCH_TIME_LIMIT = 300  # upload batch in seconds

s3_client = boto3.client('s3')

logging.basicConfig(
   filename='s3_upload.log',
   level=logging.INFO,
   format='%(asctime)s %(levelname)s %(message)s',
)

# bactching class to handle log entries
class LogBatcher:
   def __init__(self):
      self.batch = []
      self.batch_start_time = time.time()
      self.total_uploaded = 0

   def add_log_entry(self, log_entry):
      self.batch.append(log_entry)

      # check if batch should be uploaded
      if (len(self.batch) >= BATCH_SIZE or
         time.time() - self.batch_start_time >= BATCH_TIME_LIMIT):
         self.upload_batch()

   #upload the current batch to S3
   def upload_batch(self):
      if not self.batch:
         return
```

```
    try:
        # create batch content
        batch_content = '\n'.join(json.dumps(entry) for entry in self.batch)

        # generate timestamp-based filename
        timestamp = datetime.now().strftime('%Y-%m-%d_%H-%M-%S')
        batch_id = f"{timestamp}_{len(self.batch)}_entries"


        object_key = f"suricata-logs/{batch_id}.jsonl"
        content_type = "application/json"
        body = batch_content

        # create metadata
        metadata = {
            'batch_size': str(len(self.batch)),
            'upload_timestamp': datetime.now().strftime('%Y-%m-%d %H:%M:%S'),
            'content_format': 'jsonl',  # JSON Lines format
        }

        # upload to S3
        s3_client.put_object(
            Bucket=BUCKET_NAME,
            Key=object_key,
            Body=body,
            ContentType=content_type,
            Metadata=metadata
        )

        self.total_uploaded += len(self.batch)
        logging.info(f"Uploaded batch of {len(self.batch)} entries to {object_key}")
        print(f"{datetime.now().strftime('%H:%M:%S')}: Uploaded batch:
{len(self.batch)} entries -> {object_key}")

        # reset batch
        self.batch = []
        self.batch_start_time = time.time()

        time.sleep(15)
    except Exception as e:
        logging.error(f"Error uploading batch: {e}")
        print(f"Error uploading batch: {e}")
```

```
    # upload any remaining entries
    def flush_remaining(self):
        if self.batch:
            self.upload_batch()

# function to process each event
def process_event(event_json, batcher):
    try:
        # parse the event
        event = json.loads(event_json) if isinstance(event_json, str) else event_json

        # add processing timestamp
        event['_processed_at'] = datetime.now().isoformat()

        #add to batch instead of uploading individually
        batcher.add_log_entry(event)
        return True

    except json.JSONDecodeError as e:
        logging.error(f"Error decoding JSON: {e}")
        return False
    except Exception as e:
        logging.error(f"Unexpected error in process_event: {e}")
        return False

# function to monitor the log file and process new entries
def monitor_log(file_path):
    current_position = 0
    batcher = LogBatcher()

    print(f"Monitoring {file_path} for new log entries...")
    print(f"Batch size: {BATCH_SIZE} entries")
    print(f"Batch time limit: {BATCH_TIME_LIMIT} seconds")

    if not os.path.exists(file_path):
        logging.error(f"File {file_path} does not exist.")
        raise FileNotFoundError(f"The file {file_path} could not be found")

    try:
        while True:
            try:
                file_size = os.path.getsize(file_path)

                if file_size > current_position:
```

```
            with open(file_path, 'r') as f:
                f.seek(current_position)
                new_lines = f.readlines()

                if new_lines:
                    processed_lines = 0
                    for line in new_lines:
                        if not line.strip():
                            continue
                        try:
                            if process_event(line, batcher):
                                processed_lines += 1
                        except Exception as e:
                            logging.error(f"Error processing line: {e}")

                    logging.info(f"Processed {processed_lines} new lines from {file_path}.")

                current_position = f.tell()

            elif file_size < current_position:
                logging.info("Log file size decreased, resetting position.")
                # upload any pending batch before resetting
                batcher.flush_remaining()
                current_position = 0
            else:
                # no new entries, but check if we need to upload due to time limit
                if (batcher.batch and
                    time.time() - batcher.batch_start_time >= BATCH_TIME_LIMIT):
                    batcher.upload_batch()

        except FileNotFoundError:
            logging.error(f"File not found: {file_path}")
            print(f"File not found: {file_path}")
            break

    except KeyboardInterrupt:
        # upload any remaining entries
        batcher.flush_remaining()
        logging.info(f"Shutdown complete. Total entries uploaded:
{batcher.total_uploaded}")

if __name__ == "__main__":
    try:
        # check if bucket exists
```

```
    s3_client.head_bucket(Bucket=BUCKET_NAME)

    # log file path
    file_path = 'eve.json' if os.path.exists('eve.json') else
'/usr/local/var/log/suricata/eve.json'
    #print(f"Using log file: {file_path}")
    monitor_log(file_path)

  except ClientError as e:
    error_code = e.response.get('Error', {}).get('Code')
    if error_code == '404':
      logging.error(f"Bucket {BUCKET_NAME} does not exist.")
      print(f"Bucket {BUCKET_NAME} does not exist.")
    else:
      logging.error(f"Error accessing bucket {BUCKET_NAME}: {e}")
      print(f"Error accessing bucket: {e}")
    exit(1)
  except Exception as e:
    logging.error(f"Error: {e}")
    print(f"Error: {e}")
    exit(1)
```

Lambda Function Code

```
import json
import boto3
from urllib.parse import unquote_plus
from datetime import datetime, timezone
from opensearchpy import OpenSearch, RequestsHttpConnection
import os

# initialize S3 client
s3_client = boto3.client('s3')

# environment variable for SNS
SNS_TOPIC_ARN = os.environ['SNS_TOPIC_ARN']

# environment variables for OpenSearch
OPENSEARCH_ENDPOINT = os.environ['OPENSEARCH_ENDPOINT']
```

```
OPENSEARCH_INDEX = os.environ['OPENSEARCH_INDEX'] if 'OPENSEARCH_INDEX'
in os.environ else 'suricata-logs'

#authentication for OpenSearch
OPENSEARCH_USERNAME = os.environ['OPENSEARCH_USERNAME']
OPENSEARCH_PASSWORD = os.environ['OPENSEARCH_PASSWORD']

def lambda_handler(event, context):
  print(f"Lambda invoked with event: {json.dumps(event)}")

  sns = boto3.client('sns')

  # set minimum severity level for notifications (critical, high, medium, or low)
  MIN_SEVERITY_LEVEL = 'critical'

  # initialize OpenSearch client if endpoint is configured
  opensearch_client = None
  if OPENSEARCH_ENDPOINT:
    opensearch_client = initialize_opensearch_client()

  for record in event['Records']:
    # extract S3 event details
    bucket_name = record['s3']['bucket']['name']
    object_key = unquote_plus(record['s3']['object']['key'])
    object_size = record['s3']['object']['size']
    event_name = record['eventName']
    event_time = record['eventTime']
    region = record['awsRegion']

    # parse the object key for meaningful information
    key_parts = object_key.split('/')

    # format timestamp
    formatted_time = datetime.fromisoformat(event_time.replace('Z',
'+00:00')).strftime('%Y-%m-%d %H:%M:%S UTC')

    if object_key.lower().endswith('.jsonl'):
      try:
        # process the JSONL file
        file_content = process_file(bucket_name, object_key)

        if opensearch_client:
        # process JSONL files for OpenSearch ingestion
```

```
            ingest_to_opensearch(opensearch_client, bucket_name,
object_key,file_content, context.aws_request_id)

          # read the log content for metadata severity
          if 'suricata' in object_key.lower():
            alerts = get_suricata_alerts(file_content, MIN_SEVERITY_LEVEL)

            # if no alerts found that meet severity threshold, skip
            if not alerts:
              print(f"No alerts found meeting {MIN_SEVERITY_LEVEL} severity
threshold")
              continue

            # process each qualifying alert
            for alert_data in alerts:
              severity = alert_data['severity']
              log_data = alert_data['log_data']

              # send alert for this specific log entry
              send_suricata_alert(sns, SNS_TOPIC_ARN, severity, log_data,
bucket_name, object_key, formatted_time, object_size)
              print(f"Test: Sending alert for log entry: {log_data}")

        except Exception as e:
          print(f"Error processing JSONL file {object_key}: {str(e)}")
          continue
      else:
        # use path-based detection for non-suricata logs
        severity = determine_severity_from_path(object_key)

        # check if this severity meets threshold
        if not should_send_alert(severity, MIN_SEVERITY_LEVEL):
          continue

        # send generic alert
        send_generic_alert(sns, SNS_TOPIC_ARN, severity, bucket_name, object_key,
formatted_time, object_size, event_name, region)
        print(f"Test: Sending generic alert for log entry: {object_key}")

    return {
      'statusCode': 200,
      'body': json.dumps('Successfully processed S3 events with severity filtering')
    }
```

```
# function to read suricata JSONL file from S3 and extract alerts that meet severity threshold
def get_suricata_alerts(content, min_severity):
  try:
    qualifying_alerts = []
    line_num = 0

    # process each line in the JSONL file
    for line in content.split('\n'):
      line_num += 1
      line = line.strip()

      # skip empty lines
      if not line:
        continue

      try:
        # parse JSON log entry
        log_data = json.loads(line)

        # extract severity from metadata
        alert = log_data.get('alert', {})
        metadata = alert.get('metadata', {})

        severity = None
        if 'severity' in metadata and metadata['severity']:
          severity_list = metadata['severity']
          if isinstance(severity_list, list) and severity_list:
            severity = severity_list[0].lower()

        # if severity found and meets threshold, add to results
        if severity and should_send_alert(severity, min_severity):
          qualifying_alerts.append({
            'severity': severity,
            'log_data': log_data,
            'line_number': line_num
          })

      except json.JSONDecodeError as e:
        print(f"Error parsing JSON on line {line_num}: {str(e)}")
        continue

    return qualifying_alerts
```

```
    except Exception as e:
        print(f"Error reading JSONL file from S3: {str(e)}")
        raise e

# function to send Suricata-specific alert
def send_suricata_alert(sns, topic_arn, severity, log_data, bucket_name,
object_key,
                        formatted_time, object_size):
    try:
        alert_info = log_data.get('alert', {})

        severity_emoji = get_severity_emoji(severity)
        subject = f"{severity_emoji} Suricata Alert [{severity.upper()}]:
{alert_info.get('signature', 'Unknown')}"

        message = f"""
{severity_emoji} SECURITY ALERT - Suricata Detection

🔍 SEVERITY: {severity.upper()}
🚨 Signature: {alert_info.get('signature', 'Unknown')}
📋 Category: {alert_info.get('category', 'Unknown')}
🆔 Rule ID: {alert_info.get('signature_id', 'Unknown')}
⚡ Action: {alert_info.get('action', 'Unknown')}

Network Details:
• Source: {log_data.get('src_ip', 'Unknown')}:{log_data.get('src_port', 'Unknown')}
• Destination: {log_data.get('dest_ip', 'Unknown')}:{log_data.get('dest_port',
'Unknown')}
• Protocol: {log_data.get('proto', 'Unknown')}
• Interface: {log_data.get('in_iface', 'Unknown')}
• Flow ID: {log_data.get('flow_id','Unknown')}

File Details:
• Bucket: {bucket_name}
• File: {object_key.split('/')[-1]}
• Time: {formatted_time}
• Size: {object_size} bytes

Full Path: s3://{bucket_name}/{object_key}

        """.strip()

        print(f"Sending {severity} priority alert to SNS: {subject}")
```

```
        response = sns.publish(
            TopicArn=topic_arn,
            Message=message,
            Subject=subject
        )
        print(f"Successfully sent SNS notification: {response['MessageId']}")

    except Exception as e:
        print(f"Error sending Suricata SNS notification: {str(e)}")
        raise e

# function to send generic alert
def send_generic_alert(sns, topic_arn, severity, bucket_name, object_key,
                       formatted_time, object_size, event_name, region):
    try:
        severity_emoji = get_severity_emoji(severity)
        subject = f"{severity_emoji} Alert [{severity.upper()}]: {bucket_name}"

        message = f"""
{severity_emoji} File Alert - {severity.upper()} Priority

Bucket: {bucket_name}
File: {object_key.split('/')[-1]}
Full Path: {object_key}
Size: {format_bytes(object_size)}
Time: {formatted_time}
Event: {event_name}
Region: {region}

S3 Location: s3://{bucket_name}/{object_key}

Severity: {severity.upper()}
        """.strip()

        print(f"Sending {severity} priority alert to SNS: {subject}")
        response = sns.publish(
            TopicArn=topic_arn,
            Message=message,
            Subject=subject
        )
        print(f"Successfully sent SNS notification: {response['MessageId']}")

    except Exception as e:
        print(f"Error sending generic SNS notification: {str(e)}")
```

```
        raise e

# function to determine severity based on file path or content
def determine_severity_from_path(object_key):
    object_key_lower = object_key.lower()

    # check for common severity indicators in the object key
    if 'critical' in object_key_lower:
        return 'critical'

    if 'high' in object_key_lower:
        return 'high'

    if 'medium' in object_key_lower:
        return 'medium'

    if 'low' in object_key_lower:
        return 'low'

    # default severity if no indicators found
    return 'low'

# function to determine if an alert should be sent based on severity
def should_send_alert(detected_severity, min_severity):
    # Create severity hierarchy
    hierarchy = ['low', 'medium', 'high', 'critical']
    try:
        detected_level = hierarchy.index(detected_severity)
        min_level = hierarchy.index(min_severity)
        return detected_level >= min_level
    except ValueError:
        # if severity not found in hierarchy, default to sending alert
        return True

# function to get severity emoji based on severity level
def get_severity_emoji(severity):
    emoji_map = {
        'critical': '🚨',
        'high': '⚠️',
        'medium': '⚡',
        'low': 'ℹ️'
    }
    return emoji_map.get(severity.lower(), '📋')
```

```
# function to format bytes into a human-readable string
def format_bytes(bytes_size):
  for unit in ['B', 'KB', 'MB', 'GB']:
    if bytes_size < 1024.0:
      return f"{bytes_size:.1f} {unit}"
    bytes_size /= 1024.0
  return f"{bytes_size:.1f} TB"

# read S3 file and return its content
def process_file(bucket_name, object_key):
  try:
    response = s3_client.get_object(Bucket=bucket_name, Key=object_key)
    content = response['Body'].read().decode('utf-8').strip()
    return content
  except Exception as e:
    print(f"Error reading S3 file {object_key}: {str(e)}")
    raise e

# function to initialize OpenSearch client
def initialize_opensearch_client():
  try:

    if not OPENSEARCH_USERNAME or not OPENSEARCH_PASSWORD:
      print("OpenSearch credentials are not set in environment variables.")
      return None

    # create OpenSearch client
    client = OpenSearch(
      hosts=[{'host': OPENSEARCH_ENDPOINT.replace('https://', ''), 'port': 443}],
      http_auth=(OPENSEARCH_USERNAME, OPENSEARCH_PASSWORD),
      use_ssl=True,
      verify_certs=True,
      connection_class=RequestsHttpConnection,
      timeout=60
    )

    info = client.info()
    print(f"OpenSearch client initialized successfully: {info['cluster_name']}")
    return client
  except Exception as e:
    print(f"Failed to initialize OpenSearch client: {str(e)}")
    return None
```

```
# function to ingest JSONL content to OpenSearch
def ingest_to_opensearch(client, bucket_name, object_key, content, request_id):
  try:
    # process JSONL content
    logs = []
    line_num = 0
    for line in content.split('\n'):
      line_num += 1
      line = line.strip()

      # skip empty lines
      if not line:
        continue

      try:
        # parse JSON log entry
        log_data = json.loads(line)

        # add metadata for tracking
        log_data['_source_file'] = object_key
        log_data['_source_bucket'] = bucket_name
        log_data['_ingestion_timestamp'] = datetime.now(timezone.utc).isoformat()

        # generate document ID
        log_id = f"{object_key.replace('/', '_')}_{line_num}"

        # append to documents list
        logs.append({
          '_id': log_id,
          '_source': log_data
        })

      except json.JSONDecodeError as e:
        print(f"JSON decode error at line {line_num} in {object_key}: {e}")
        continue
    # check if logs were found
    if not logs:
      print(f"No valid JSON documents found in {object_key}")
      return

    # bulk index to OpenSearch
    success_count = bulk_index_documents(client, logs)
    print(f"Successfully indexed {success_count}/{len(logs)} documents from
{object_key}")
```

```
    except Exception as e:
        print(f"Error ingesting {object_key} to OpenSearch: {str(e)}")
        raise e

# function to bulk index documents to OpenSearch
def bulk_index_documents(client, logs):
    events = []

    for log in logs:
        # create index action for each log
        index_action = {
            "index": {
                "_index": OPENSEARCH_INDEX,
                "_id": log['_id']
            }
        }
        events.append(json.dumps(index_action))
        events.append(json.dumps(log['_source']))

    if not events:
        return 0

    # prepare bulk request body
    bulk_body = '\n'.join(events) + '\n'

    try:
        # execute bulk request
        response = client.bulk(body=bulk_body)

        # count successful operations
        success_count = 0
        if 'items' in response:
            for item in response['items']:
                if 'index' in item and item['index'].get('status') in [200, 201]:
                    success_count += 1
                elif 'index' in item:
                    print(f"Failed to index document: {item['index']}")

        return success_count

    except Exception as e:
        print(f"Bulk indexing failed: {str(e)}")
        raise e
```

Suricata Custom Rules

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"Potential TCP SYN flood"; flow:to_client; flags: S,12; threshold: type both, track by_dst, count 200, seconds 5; classtype:attempted-dos; sid:5000010;rev:1; metadata:severity medium;)

alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"Potential TCP SYN flood High"; flow:established; flags: S,12; threshold: type both, track by_dst, count 500, seconds 5; classtype:attempted-dos; sid:5000011;rev:1; metadata:severity high;)

alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"Potential TCP RST flood"; flow:established; flags: R; threshold: type both, track by_dst, count 200, seconds 5; classtype:attempted-dos; sid:5000012;rev:1; metadata:severity medium;)

alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"Potential TCP RST flood High"; flow:established; flags: R; threshold: type both, track by_dst, count 500, seconds 5; classtype:attempted-dos; sid:5000013;rev:1; metadata:severity high;)

alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"Potential TCP FNH flood"; flow:established; flags: F; threshold: type both, track by_dst, count 100, seconds 5; classtype:attempted-dos; sid:5000014;rev:1; metadata:severity medium;)

alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"Potential TCP SYN FNH High"; flow:established; flags: F; threshold: type both, track by_dst, count 200, seconds 5; classtype:attempted-dos; sid:5000015;rev:1; metadata:severity high;)

alert tcp $EXTERNAL_NET any -> $HOME_NET 21 (msg:"FTP Brute Force attempt"; flow:to_server,established; content:"USER "; nocase; threshold: type threshold, track by_src, count 5, seconds 60; classtype:attempted-recon; sid:5000016; rev:1;)

alert tcp $EXTERNAL_NET any -> $HOME_NET 20 (msg:"FTP Brute Force data transfer attempt"; flow:to_server,established; content:"USER "; nocase; threshold: type threshold, track by_src, count 5, seconds 60; classtype:attempted-recon; sid:5000017; rev:1;)

alert tcp $EXTERNAL_NET any -> $HOME_NET 22 (msg:"SSH Brute Force Attempt"; flags:S; threshold: type threshold, track by_src, count 10, seconds 60; classtype:attempted-recon; sid:5000018; rev:1;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"Potential HTTP GET flood"; flow:to_server,established; http.method; content:"GET"; threshold: type both, track
```

```
by_src, count 50, seconds 5; classtype:attempted-dos; sid:5000200; rev:1;
metadata:severity medium;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"Potential HTTP GET flood
High"; flow:to_server,established; http.method; content:"GET"; threshold: type
both, track by_src, count 100, seconds 5; classtype:attempted-dos; sid:50000201;
rev:1; metadata:severity high;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"API brute force - repeated
requests"; flow:to_server,established; content:"POST"; http.method;
content:"/api/auth"; http.uri;
pcre:"/\/(api\/auth|v1\/login|auth\/login|oauth\/token)/Ui"; threshold:type both,
track by_src, count 50, seconds 5; classtype:attempted-recon; sid:5000202; rev:1;
metadata:severity medium;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"API brute force - repeated
requests High"; flow:to_server,established; content:"POST"; http.method;
content:"/"; http.uri; pcre:"/\/(api\/auth|v1\/login|auth\/login|oauth\/token)/Ui";
threshold:type both, track by_src, count 100, seconds 5; classtype:attempted-
recon; sid:5000203; rev:1; metadata:severity high;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"Potential HTTP POST
flood"; flow:to_server,established; http.method; content:"POST"; threshold: type
both, track by_src, count 50, seconds 5; classtype:attempted-dos; sid:50000204;
rev:1; metadata:severity medium;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"Potential HTTP POST flood
High"; flow:to_server,established; http.method; content:"POST"; threshold: type
both, track by_src, count 100, seconds 5; classtype:attempted-dos; sid:50000205;
rev:1; metadata:severity high;)

alert http $EXTERNAL_NET any -> any any (msg:"Web Server Enumeration Attempt";
urilen:>100; threshold:type threshold, track by_src, count 10, seconds 60;
classtype:web-application-attack; sid:50000206; rev:1; metadata:severity medium;)

alert http $EXTERNAL_NET any -> $HOME_NET any ( msg:"Potential XSS Attack
Attempt";flow: to_server,established; content:"<script"; nocase; http_uri;
pcre:"/<script.*?>/i";classtype:web-application-attack; sid:50000207; rev:1;
metadata:severity high;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"SQLi Attempt - Pattern-
based in URI"; flow:to_server,established; content:"/"; http.uri;
pcre:"/('|%27|%22)(\s*(or|and)\s+|\s*=\s*|\s*--|\s*#|\d+=\d+)/i"; classtype:web-
application-attack; sid:50000208; rev:2; metadata:severity high;)
```

```
alert http $EXTERNAL_NET any -> $HOME_NET any ( msg:"SQLi Attempt - POST body"; flow:to_server,established; content:"application/x-www-form-urlencoded"; http_header; content:"' OR 1=1"; nocase; http_client_body; classtype:web-application-attack; sid:50000209; rev:1; metadata:severity high;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"SQLi Attempt - UNION in URI"; flow:to_server,established; content:"/"; http.uri; pcre:"/(union(\+|%20)?select)/i"; classtype:web-application-attack; sid:50000210; rev:1; metadata:severity high;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"SQLi Attempt - SELECT/DROP in URI"; flow:to_server,established; content:"/"; http.uri; pcre:"/(select\s+\*|drop\s+table|insert\s+into|update\s+\w+\s+set)/i"; classtype:web-application-attack; sid:50000211; rev:1; metadata:severity high;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg: "Possible SQL Injection attack (Contains singlequote)"; flow:established,to_server; content:"'"; nocase; http_uri;classtype:web-application-attack; sid:50000212; rev:1;metadata:severity high;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg: "Possible SQL Injection attack (Contains UNION)"; flow:established,to_server; content:"union"; nocase; http_uri; classtype:web-application-attack;sid:50000213; rev:1;metadata:severity high;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg: "Possible SQL Injection attack (Contains SELECT)"; flow:established,to_server; content:"select"; nocase; http_uri; classtype:web-application-attack;sid:50000214; rev:1;metadata:severity high;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg: "Possible SQL Injection attack (Contains singlequote POST DATA)"; flow:established,to_server; content:"'"; nocase; http_client_body;classtype:web-application-attack; sid:50000216; rev:1;metadata:severity high;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg: "Possible SQL Injection attack (Contains UNION POST DATA)"; flow:established,to_server; content:"union"; nocase; http_client_body; classtype:web-application-attack;sid:50000217; rev:1;metadata:severity high;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg: "Possible SQL Injection attack (Contains SELECT POST DATA)"; flow:established,to_server; content:"select"; nocase; http_client_body; classtype:web-application-attack;sid:50000218; rev:1;metadata:severity high;)
```

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"Authentication Bypass Attempt "; content:"Admin=true"; classtype:web-application-attack;sid:50000219;rev:1;metadata:severity medium;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"Potential XSS Attack"; content:"<script>"; classtype:web-application-attack;sid:50000220;rev:1;metadata:severity high;)

alert icmp $EXTERNAL_NET any -> $HOME_NET any (msg:"Potential ICMP flood"; threshold: type both, track by_dst, count 50, seconds 5; classtype:attempted-dos; sid:50000030; rev:1; metadata:severity medium;)

alert icmp $EXTERNAL_NET any -> $HOME_NET any (msg:"Potential ICMP flood High"; threshold: type both, track by_dst, count 100, seconds 5; classtype:attempted-dos; sid:50000031; rev:1; metadata:severity high;)

alert udp $EXTERNAL_NET any -> $HOME_NET any (msg:"Potential UDP flood"; threshold: type both, track by_dst, count 50, seconds 5; classtype:attempted-dos; sid:5000040; rev:1; metadata:severity medium;)

alert udp $EXTERNAL_NET any -> $HOME_NET any (msg:"Potential UDP flood High"; threshold: type both, track by_dst, count 100, seconds 5; classtype:attempted-dos; sid:5000041; rev:1; metadata:severity high;)

alert ip $EXTERNAL_NET any -> $HOME_NET any (msg:"Potential IP flood"; threshold: type both, track by_src, count 5000, seconds 60; classtype:attempted-dos; sid:5000050;rev:1; metadata:severity medium;)

alert ip $EXTERNAL_NET any -> $HOME_NET any (msg:"Potential IP flood High"; threshold: type both, track by_src, count 10000, seconds 60; classtype:attempted-dos; sid:5000051;rev:1; metadata:severity high;)

alert http any any -> $HOME_NET any (msg:"Suspicious executable download from Dropbox"; flow:established,to_server; http.host; content:"dropbox.com"; http.uri; content:".exe"; nocase; endswith; classtype:suspicious-filename-detect; sid:5000060; rev:1; metadata:severity high;)

alert http any any -> $HOME_NET any (msg:"Suspicious file download from Dropbox"; flow:established,to_server; http.host; content:"dropbox.com"; http.method; content:"GET"; http.uri; content:"/"; pcre:"/\.(exe|dll|bat|com|cmd|scr|jar|js|ps1|wsh|vbe|vbs|sh)$/i"; classtype:suspicious-filename-detect; sid:5000061; rev:1; metadata:severity high;)

alert tcp any any -> any any (msg:"Potential malware execution"; flow:established,from_server; content:"|4D 5A|"; depth:2; content:"|50 45 00 00|";
```

```
distance:58; within:4; classtype:trojan-activity; sid:5000062; rev:1;metadata:
severity high;)

alert http any any -> $HOME_NET any (msg:"Suspicious tool download from
Dropbox"; flow:established,to_server; http.host; content:"dropbox.com"; http.uri;
pcre:"/\.(zip|rar|7z|tar|gz)$/i"; classtype:suspicious-filename-detect; sid:5000063;
rev:1; metadata:severity medium;)

# Web Attack - Brute Force (9:20-10:00 AM)

# Lower threshold counts for more sensitive detection of the described attack

alert http 205.174.165.73 any -> $HOME_NET any (msg:"Specified - API brute force -
Specific attack pattern 2017-07-06"; flow:to_server,established; content:"POST";
http.method; content:"/"; http.uri;
pcre:"/\/(api\/auth|v1\/login|auth\/login|oauth\/token)/Ui"; threshold:type both,
track by_src, count 25, seconds 5; classtype:attempted-recon; sid:5000950; rev:1;
metadata:severity high;)

# Web Attack - XSS (10:15-10:35 AM)

# More specific XSS detection rules targeting the attack vectors used

alert http 205.174.165.73 any -> $HOME_NET any (msg:"Specified - XSS Attack 2017-
07-06"; flow:to_server,established; content:"<script"; nocase; http_uri;
pcre:"/<script.*?>/i"; classtype:web-application-attack; sid:5000951; rev:1;
metadata:severity high;)

alert http 205.174.165.73 any -> $HOME_NET any (msg:"Specified - XSS Attack via
POST 2017-07-06"; flow:to_server,established; content:"<script"; nocase;
http_client_body; pcre:"/<script.*?>/i"; classtype:web-application-attack;
sid:5000952; rev:1; metadata:severity high;)

# Web Attack - SQL Injection (10:40-10:42 AM)

# More tailored SQL injection rules for the specific attack

alert http 205.174.165.73 any -> $HOME_NET any (msg:"Specified - SQLi Attack 2017-
07-06 URI"; flow:to_server,established; content:"/"; http.uri;
pcre:"/('|%27|%22)(\s*(or|and)\s+|\s*=\s*|\s*--|\s*#|\d+=\d+)/i"; classtype:web-
application-attack; sid:5000953; rev:1; metadata:severity high;)

alert http 205.174.165.73 any -> $HOME_NET any (msg:"Specified - SQLi Attack 2017-
07-06 UNION"; flow:to_server,established; content:"/"; http.uri;
pcre:"/(union(\+|%20)?select)/i"; classtype:web-application-attack; sid:5000954;
rev:1; metadata:severity high;)
```

```
# --- INFILTRATION RULES (AFTERNOON) ---

# Dropbox Download - Metasploit Exploit (14:19-14:21 & 14:33-14:35)

alert http any any -> $HOME_NET any (msg:"Specified - Infiltration - Suspicious
executable from Dropbox 2017-07-06"; flow:established,to_server; http.host;
content:"dropbox.com"; http.uri; content:".exe"; nocase; endswith;
classtype:suspicious-filename-detect; sid:5000955; rev:1; metadata:severity high;)

# Explicitly monitoring traffic from attacker IP

alert http 205.174.165.73 any -> $HOME_NET any (msg:"Specified - Potential
malicious download from attacker 2017-07-06"; flow:established,to_client;
classtype:suspicious-filename-detect; sid:5000956; rev:1; metadata:severity high;)

# Cool Disk - MAC Infiltration (14:53-15:00)

alert http any any -> 192.168.10.25 any (msg:"Specified - MAC Infiltration attempt
2017-07-06"; flow:established,to_server; http.host; content:"dropbox.com";
http.uri; pcre:"/\.(dmg|pkg)$/i"; classtype:suspicious-filename-detect; sid:5000957;
rev:1; metadata:severity high;)

# Second Dropbox Download - Vista (15:04-15:45)

alert http any any -> 192.168.10.8 any (msg:"Specified - Vista Infiltration attempt
2017-07-06"; flow:established,to_server; http.host; content:"dropbox.com";
classtype:suspicious-filename-detect; sid:5000958; rev:1; metadata:severity high;)

# Port Scan detection from compromised Vista

alert tcp 192.168.10.8 any -> $HOME_NET any (msg:"Specified - Port scan from
compromised Vista 2017-07-06"; flow:to_server; flags:S; threshold:type threshold,
track by_src, count 25, seconds 60; classtype:attempted-recon; sid:5000959; rev:1;

   metadata:severity high;)

# Nmap scan detection from compromised Vista

alert tcp 192.168.10.8 any -> $HOME_NET any (msg:"Specified - Nmap scan from
compromised Vista 2017-07-06"; flow:to_server; flags:S,12; window:1024;
tcp.mss:1460; threshold:type threshold, track by_src, count 15, seconds 60;
classtype:attempted-recon; sid:5000960; rev:1;

   metadata:severity high;)

# --- NAT TRAVERSAL SPECIFIC RULES ---
```

```
# Track attacks attempting to traverse the NAT

alert tcp any any -> 205.174.165.80 any (msg:"Specified - Attack attempt through firewall NAT"; flow:to_server; classtype:attempted-admin; sid:5000961; rev:1; metadata:severity high;)

# --- CONSOLIDATED ATTACK PATTERN DETECTION ---

# Consolidated rule to detect the entire attack pattern

alert ip 205.174.165.73 any -> $HOME_NET any (msg:"Specified - Detected attack pattern from Kali attacker"; threshold: type both, track by_src, count 15, seconds 60; classtype:attempted-admin; sid:5000962; rev:1; metadata:severity high;)

#------

alert http any any -> $HOME_NET any (msg:"Network scanning tool download from Dropbox"; flow:established,to_server; http.host; content:"dropbox.com"; http.uri; content:"nmap"; nocase; classtype:suspicious-filename-detect; sid:2000006; rev:1; metadata:severity high;)

alert tcp $EXTERNAL_NET any -> any any (msg:"Port scan activity"; flow:to_server; flags:S; threshold:type threshold, track by_src, count 30, seconds 60; classtype:attempted-recon; sid:5000070; rev:1;metadata: severity medium;)

alert tcp $EXTERNAL_NET any -> any any (msg:"Nmap SYN scan detected"; flow:to_server; flags:S,12; window:1024; tcp.mss:1460; threshold:type threshold, track by_src, count 15, seconds 60; classtype:attempted-recon; sid:5000071; rev:1; metadata:severity medium;)

alert tcp $EXTERNAL_NET any -> any any (msg:"Nmap FIN scan detected"; flow:to_server; flags:F,12; window:1024; tcp.mss:1460; threshold:type threshold, track by_src, count 15, seconds 60; classtype:attempted-recon; sid:5000072; rev:1;metadata: severity medium;)

alert tcp $EXTERNAL_NET any -> any any (msg:"Nmap NULL scan detected"; flow:to_server; flags:0,12; window:1024; tcp.mss:1460; threshold:type threshold, track by_src, count 15, seconds 60; classtype:attempted-recon; sid:5000073; rev:1;metadata: severity medium;)

alert tcp $EXTERNAL_NET any -> any any (msg:"Nmap XMASS scan detected"; flow:to_server; flags:FA,12; window:1024; tcp.mss:1460; threshold:type threshold, track by_src, count 15, seconds 60; classtype:attempted-recon; sid:5000074; rev:1;metadata: severity medium;)
```

```
alert http any any -> $HOME_NET any (msg:"Suspicious Mac disk image download from Dropbox"; flow:established,to_server; http.host; content:"dropbox.com"; http.uri; content:"/"; pcre:"/\.(dmg|pkg)$/i"; classtype:suspicious-filename-detect; sid:5000075; rev:1; metadata:severity medium;)

alert tcp any any -> any any (msg:"Port scan from potentially compromised host"; flow:to_server; flags:S; threshold:type threshold, track by_src, count 50, seconds 60; classtype:attempted-recon; sid:5000076; rev:1;metadata: severity high;)

alert tcp $EXTERNAL_NET any -> any any (msg:"Nmap SYN scan detected"; flow:to_server; flags:S,12; window:1024; tcp.mss:1460; threshold:type threshold, track by_src, count 20, seconds 60; classtype:attempted-recon; sid:5000077; rev:1;metadata: severity high;)

alert tcp $EXTERNAL_NET any -> $HOME_NET 80 (msg:"Possible Slowloris DoS attack"; flow:established,to_server; content:"GET"; depth:4; content:"HTTP"; distance:0; pcre:"/(?:^|[^\r])$/"; threshold:type both, track by_src, count 20, seconds 60; classtype:attempted-dos; sid:5000080; rev:1;metadata: severity high;)

alert tcp $EXTERNAL_NET any -> $HOME_NET 80 (msg:"Slowloris incomplete headers pattern"; flow:established,to_server; content:"X-"; nocase; depth:2; threshold:type both, track by_src, count 30, seconds 120; classtype:attempted-dos; sid:5000081; rev:1;metadata: severity high;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"Slowloris similar User-Agent pattern"; flow:established,to_server; content:"User-Agent"; http_header; threshold:type both, track by_src, count 50, seconds 60; flowbits:set,slowloris; classtype:attempted-dos; sid:3000003; rev:1;metadata: severity high;)

alert tcp $EXTERNAL_NET any -> $HOME_NET 80 (msg:"Slowhttptest Headers DoS attempt"; flow:established,to_server; content:"GET"; depth:4; threshold:type both, track by_src, count 30, seconds 60; classtype:attempted-dos; sid:5000082; rev:1;metadata: severity high;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"Slowhttptest Slow POST DoS attempt"; flow:established,to_server; content:"POST"; http_method; content:"Content-Length:"; http_header; pcre:"/Content-Length: [0-9]{4,}/i"; threshold:type both, track by_src, count 15, seconds 60; classtype:attempted-dos; sid:5000083; rev:1;metadata: severity high;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"Slowhttptest tool signature"; flow:established,to_server; content:"SlowHTTPTest"; nocase; http_user_agent; classtype:attempted-dos; sid:5000084; rev:1;metadata: severity high;)
```

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"HULK DoS attack pattern"; flow:established,to_server; content:"?"; http_uri; pcre:"/\?[a-zA-Z0-9]{4,}=[a-zA-Z0-9]{4,}&[a-zA-Z0-9]{4,}=[a-zA-Z0-9]{4,}/U"; threshold:type both, track by_src, count 30, seconds 30; classtype:attempted-dos; sid:5000085; rev:1;metadata: severity high;)

alert http $EXTERNAL_NET any -> any any (msg:"HULK cache-busting pattern"; flow:established,to_server; content:"GET"; http_method; content:"cache="; http_uri; distance:0; pcre:"/cache=[0-9]+/i"; threshold:type both, track by_src, count 25, seconds 30; classtype:attempted-dos; sid:5000086; rev:1;metadata: severity high;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"HULK no-cache pattern"; flow:established,to_server; content:"Cache-Control: no-cache"; http_header; threshold:type both, track by_src, count 20, seconds 30; classtype:attempted-dos; sid:5000087; rev:1;metadata: severity high;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"GoldenEye DoS attack pattern"; flow:established,to_server; content:"keep-alive"; http_header; content:"Mozilla/4.0"; http_user_agent; pcre:"/[A-Za-z0-9]{4,10}: [A-Za-z0-9]{4,20}/H"; threshold:type both, track by_src, count 25, seconds 30; classtype:attempted-dos; sid:5000088; rev:1;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"GoldenEye random user-agent pattern"; flow:established,to_server; content:"User-Agent:"; http.header; pcre:"/User-Agent: Mozilla\/[0-9]\.[0-9] \([A-Za-z0-9]+;/i"; http.header; threshold:type both, track by_src, count 20, seconds 30; classtype:attempted-dos; sid:5000089; rev:1;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"GoldenEye Accept header pattern";flow:established,to_server;content:"Accept:"; http.header; content:"text/html,application/xhtml+xml,application/xml"; content:"q=0.9,*/*;q=0.8"; distance:0; within:40; threshold:type both, track by_src, count 20, seconds 30; classtype:attempted-dos; sid:50000810; rev:1;)

alert udp any any -> any any (msg:"Service discovery scan"; threshold:type threshold, track by_src, count 150, seconds 30; classtype:attempted-recon; sid:50000820; rev:1;)

#alert http any any -> any any (msg:"Attempt to access /bin*"; content:"bin"; nocase; pcre:"/(\/|%2F)bin/i"; classtype:attempted-recon; sid:50200001; rev:1;)

#alert http any any -> any any (msg:"Attempt to access /sbin*"; content:"sbin"; nocase; pcre:"/(\/|%2F)sbin/i"; classtype:attempted-recon; sid:50200002; rev:1;)
```

```
#alert http any any -> any any (msg:"Attempt to access /etc*"; content:"etc";
nocase; pcre:"/(\/|%2F)etc/i"; classtype:attempted-recon; sid:50200003; rev:1;)

#alert http any any -> any any (msg:"Attempt to access /dev*"; content:"dev";
nocase; pcre:"/(\/|%2F)dev/i"; classtype:attempted-recon; sid:50200004; rev:1;)

#alert http any any -> any any (msg:"Attempt to access /proc*"; content:"proc";
nocase; pcre:"/(\/|%2F)proc/i"; classtype:attempted-recon; sid:50200005; rev:1;)

#alert http any any -> any any (msg:"Attempt to access /var*"; content:"var";
nocase; pcre:"/(\/|%2F)var/i"; classtype:attempted-recon; sid:50200006; rev:1;)

#alert http any any -> any any (msg:"Attempt to access /tmp*"; content:"tmp";
nocase; pcre:"/(\/|%2F)tmp/i"; classtype:attempted-recon; sid:50200007; rev:1;)

#alert http any any -> any any (msg:"Attempt to access /usr*"; content:"usr";
nocase; pcre:"/(\/|%2F)usr/i"; classtype:attempted-recon; sid:50200008; rev:1;)

#alert http any any -> any any (msg:"Attempt to access /home*"; content:"home";
nocase; pcre:"/(\/|%2F)home/i"; classtype:attempted-recon; sid:50200009; rev:1;)

#alert http any any -> any any (msg:"Attempt to access /boot*"; content:"boot";
nocase; pcre:"/(\/|%2F)boot/i"; classtype:attempted-recon; sid:50200010; rev:1;)

#alert http any any -> any any (msg:"Attempt to access /lib*"; content:"lib"; nocase;
pcre:"/(\/|%2F)lib/i"; classtype:attempted-recon; sid:50200011; rev:1;)

#alert http any any -> any any (msg:"Attempt to access /opt*"; content:"opt";
nocase; pcre:"/(\/|%2F)opt/i"; classtype:attempted-recon; sid:50200012; rev:1;)

#alert http any any -> any any (msg:"Attempt to access /mnt*"; content:"mnt";
nocase; pcre:"/(\/|%2F)mnt/i"; classtype:attempted-recon; sid:50200013; rev:1;)

#alert http any any -> any any (msg:"Attempt to access /media*"; content:"media";
nocase; pcre:"/(\/|%2F)media/i"; classtype:attempted-recon; sid:50200014; rev:1;)

#alert http any any -> any any (msg:"Attempt to access /srv*"; content:"srv";
nocase; pcre:"/(\/|%2F)srv/i"; classtype:attempted-recon; sid:50200015; rev:1;)

#alert http any any -> any any (msg:"Attempt to access /root*"; content:"root";
nocase; pcre:"/^(\/|%2F)root/i"; classtype:attempted-recon; sid:50200016; rev:1;)

#alert http any any -> any any (msg:"Possible XSS attack, script tag";
content:"script"; nocase; pcre:"/(<|%3C|%253C)script/smi"; classtype:web-
application-attack; sid:50100001; rev:1;)
```

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"Possible XSS attack, js event handler"; content:"on"; nocase; pcre:"/on\w+(%3D|=)/smi"; classtype:web-application-attack; sid:50100002; rev:1;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"Possible XSS attack, js protocol"; content:"javascript"; nocase; pcre:"/javascript(:|%3A)/smi"; classtype:web-application-attack; sid:50100003; rev:1;)

alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"msfconsole powershell response"; flow:established; content:!"<html>"; content:!"<script>"; content:"|70 6f 77 65 72 73 68 65 6c 6c 2e 65 78 65|"; http_server_body; content:"|46 72 6f 6d 42 61 73 65 36 34 53 74 72 69 6e 67|"; http_server_body; classtype:exploit-kit; sid:5016005; rev:1;)

alert tls $EXTERNAL_NET any -> $HOME_NET 444 (msg:"SURICATA TLS overflow heartbeat encountered, possible exploit attempt (heartbleed)"; flow:established; app-layer-event:tls.overflow_heartbeat_message; flowint:tls.anomaly.count,+,1; classtype:protocol-command-decode; reference:cve,2014-0160; sid:2230012; rev:1;)

alert tls $EXTERNAL_NET any -> $HOME_NET 444 (msg:"SURICATA TLS invalid heartbeat encountered, possible exploit attempt (heartbleed)"; flow:established; app-layer-event:tls.invalid_heartbeat_message; flowint:tls.anomaly.count,+,1; classtype:protocol-command-decode; reference:cve,2014-0160; sid:2230013; rev:1;)

alert tls $EXTERNAL_NET any -> $HOME_NET 444 (msg:"SURICATA TLS invalid encrypted heartbeat encountered, possible exploit attempt (heartbleed)"; flow:established; app-layer-event:tls.dataleak_heartbeat_mismatch; flowint:tls.anomaly.count,+,1; classtype:protocol-command-decode; reference:cve,2014-0160; sid:2230014; rev:1;)
```